Optimizers: choosing one

Intent

DSPy ships a dozen optimizers (teleprompters), and the most-asked DSPy question is "which one should I use?" This page is the selection guide: what each optimizer tunes, what it needs at compile time, what it costs, and when it wins against the alternatives.

Read this once you have a working program and a metric, and you're trying to decide what to compile against. The deep-dive pages on individual optimizers (GEPA, the BootstrapFewShot family, BootstrapFinetune) live alongside this one.

Design decisions

1. Every optimizer tunes one or more of: instructions, demos, or weights

The three knobs cover every search space DSPy exposes. Instructions are the natural-language docstring on each predictor's signature. Demos are the in-context examples each predictor sees at call time. Weights are the model parameters when the LM is tunable. Each optimizer picks one or two of these and leaves the rest alone; the API walkthrough below groups them by which knob they turn.

2. `.compile()` returns a new copy; the original student isn't mutated

The first thing every optimizer does is `student.reset_copy()` — a deepcopy that also clears inherited `_compiled` flags. The compiled program is returned; the original you passed in stays untouched. You can re-run `.compile()` on the same student with different optimizers, datasets, or budgets and compare results without worrying about state leak between runs.

3. `_compiled=True` is the flag that tells future optimizers to leave a sub-module alone

After a successful `.compile()`, the optimizer sets `_compiled = True` on the returned module. When you wrap that compiled module inside a bigger program and run a second optimizer on the outer one, `named_parameters()` skips the inner compiled module — its predictors stay as the first optimizer left them. The flag is what enables "optimize inner → embed in outer → optimize outer."

4. Every optimizer takes a `metric`, but the required metric shape varies

All optimizers read a numeric score per example. Most are happy with a bare float or boolean. GEPA additionally needs `Prediction(score, feedback)` because it threads the natural-language critique into its proposals. Picking the wrong metric shape for the wrong optimizer is the most common configuration mistake — check the optimizer's page when in doubt.

5. GEPA is the only optimizer that reads `Prediction(score, feedback)` per predictor

The other instruction optimizers (COPRO, MIPROv2) treat the metric as a black-box scalar — they only know whether candidate-A scored higher than candidate-B. GEPA reads `feedback` from each metric call and threads it into the next instruction proposal. The other side of the same coin: when you don't have a feedback-rich metric, COPRO or MIPROv2 may give better mileage than GEPA on the same budget.

6. Prompt-only optimizers work with any LM; finetune optimizers need a tunable model

BootstrapFewShot, COPRO, MIPROv2, GEPA, and most others mutate prompts and demos — the LM stays untouched, so closed-source providers are fair game. `BootstrapFinetune` writes new model weights and requires an LM that exposes a fine-tuning API (a local trainer, an open-source model, or a provider like OpenAI's fine-tuning endpoint). The split decides which optimizers are even available to you.

7. Demo-tuning tends to overfit; instruction-tuning tends to generalize

A demo set learned on 50 training examples describes that specific trainset. Inputs that don't resemble those examples may not benefit. An instruction rewrite, by contrast, captures patterns at a level above the examples and tends to transfer. Not a universal law, but a useful default: if your evaluation set is small or skewed, lean toward instruction optimization.

8. Most teams start prompt-only and graduate to finetune only when prompt-only plateaus

Prompt-only optimization costs LM tokens. Finetune costs LM tokens plus training compute plus deployment of new weights. The marginal lift of finetune over GEPA or MIPROv2 is usually small and sometimes negative, while the marginal cost is much larger. Treat finetune as the last lever, not the first.

9. The "compile once, save, reload" loop is what makes the optimizer cost amortize

`.compile()` is expensive — GEPA and MIPROv2 can spend hundreds of dollars in LM calls on a single run. The output is a portable artifact (`program.save(path)`); inference against it is cheap. The economics work only when you compile once per program version and serve the saved artifact many times.

10. There's no LM-based auto-selection — you choose

DSPy doesn't introspect your task or LM to pick an optimizer. The `auto` knobs on MIPROv2 and GEPA only control budget within an optimizer, not which optimizer to use. The selection cheat sheet at the bottom of this page is the closest thing to a recommendation engine.

A two-axis decision

Two questions narrow the field fast.

What's the bottleneck — instructions, demos, or weights? If the prompt wording is wrong, instruction optimizers (COPRO, GEPA, MIPROv2) help. If the model needs examples to anchor format and style, demo optimizers (BootstrapFewShot, BootstrapRS) help. If the model itself is the limit and you can fine-tune it, weight tuning (BootstrapFinetune) is the lever.

What's your compute budget? Zero-config baselines (LabeledFewShot, BootstrapFewShot) cost almost nothing. Search-based optimizers (BootstrapRS, MIPROv2, GEPA) cost real money. Combined optimizers (BetterTogether) pay both bills.

API walkthrough

Grouped by what each optimizer tunes.

Start here

The two-line baselines. Try one of these before reaching for anything heavier.

`dspy.LabeledFewShot(k=16)` No LM calls during `.compile()`. Samples up to `k` examples from the trainset (random by default, deterministic when `sample=False`) and attaches them as demos to each predictor. Reach for it as the honest baseline: if `LabeledFewShot` already gets you where you need to be, heavier optimization is wasted effort.

`dspy.BootstrapFewShot(metric=None, metric_threshold=None, teacher_settings=None, max_bootstrapped_demos=4, max_labeled_demos=16, max_rounds=1)` Runs the program (or a `teacher` you supply) on training examples, scores each completion with the metric, and keeps the traces where the metric passes. Those successful traces become the demos. The combined set mixes up to `max_bootstrapped_demos` bootstrapped traces with up to `max_labeled_demos` raw labeled examples per predictor. Almost always beats zero-shot when the metric is reliable; the safe first try.

Search across demo sets

When one bootstrap pass isn't enough.

`dspy.BootstrapFewShotWithRandomSearch(metric, max_bootstrapped_demos=4, max_labeled_demos=16, max_rounds=1, num_candidate_programs=16, num_threads=None, stop_at_score=None)` Aliased as `dspy.BootstrapRS`. Runs `BootstrapFewShot` `num_candidate_programs` times with different random seeds, evaluates each candidate on a valset, and returns the highest-scoring one. The randomness comes from which traces get bootstrapped first — different seeds discover different demo subsets. `stop_at_score` short-circuits when a candidate clears a target.

`dspy.KNNFewShot(k, trainset, vectorizer, **bootstrap_kwargs)` The demos are chosen at *inference* time, not compile time. At construction, the trainset is embedded via `vectorizer` and cached. On each call, the input is embedded, the `k` nearest training examples are retrieved, and those become the demos for that single call. Worth the indirection when no single demo set generalizes across the input distribution.

Optimize instructions

When the prompt wording — not the examples — is what's holding you back.

`dspy.COPRO(prompt_model=None, metric=None, breadth=10, depth=3, init_temperature=1.4, track_stats=False)` A breadth-first proposer. At each of `depth` levels, COPRO uses `prompt_model` to generate `breadth` candidate instructions per predictor, scores them against the trainset, and keeps the best. Total LM cost is roughly `breadth × depth × num_predictors`. Lightweight; especially good when demos are already strong and you only need to fix wording.

`dspy.GEPA(metric, auto=None, max_full_evals=None, max_metric_calls=None, reflection_lm=None, skip_perfect_score=True, instruction_proposer=None, use_merge=True, num_threads=None)` Evolutionary instruction search guided by reflection. GEPA maintains a population of programs, runs each on the trainset, reads the per-predictor `feedback` from the metric, and uses `reflection_lm` to propose edits informed by that feedback. It returns the best candidate from a Pareto frontier. Wins on prompt-only optimization when you have a strong reflection LM and a feedback-shaped metric; see GEPA in depth for the full mechanics.

Optimize instructions and demos together

`dspy.MIPROv2(metric, prompt_model=None, task_model=None, teacher_settings=None, max_bootstrapped_demos=4, max_labeled_demos=4, auto="light", num_candidates=None, num_threads=None, init_temperature=1.0, track_stats=True)` Bayesian-optimization search over the joint instruction + demo space. The `prompt_model` (often a stronger LM) proposes instructions; the `task_model` (your student) runs the program. `auto` (`"light"` / `"medium"` / `"heavy"`) translates into a budget for both proposals and evaluations. State of the art when both instructions and demos need tuning together.

`dspy.SIMBA(metric, bsize=32, num_candidates=6, max_steps=8, max_demos=4, prompt_model=None, teacher_settings=None, num_threads=None)` Mini-batch SGD-flavored search. Each step samples a `bsize` minibatch, finds the worst-scoring examples, and uses `prompt_model` to propose either a natural-language rule (instruction patch) or a new demo to address them. The mini-batch focus makes SIMBA reactive: each step targets the current weak spot rather than the average case.

`dspy.InferRules(num_candidates=10, num_rules=10, num_threads=None, **bootstrap_kwargs)` Extends `BootstrapFewShot`. After bootstrapping demos, asks a teacher LM to read them and extract `num_rules` general rules, then appends those rules to the predictor's instructions. The rules are interpretable — you can read them and decide whether to keep them. Useful when the task has patterns you want stated explicitly rather than inferred from examples.

Fine-tune the weights

`dspy.BootstrapFinetune(metric=None, multitask=True, train_kwargs=None, adapter=None, exclude_demos=False, num_threads=None)` Bootstraps successful traces the same way `BootstrapFewShot` does, then writes them as training data and fine-tunes the student LM on them. Requires an LM with a `.finetune()` method — open-source LMs through providers like Together AI, OpenAI's fine-tuning API, or a local trainer. `multitask=True` trains one model across all predictors; `False` trains a separate model per predictor.

Compose optimizers

`dspy.BetterTogether(metric, **optimizers)` A meta-optimizer that runs a sequence specified as a string like `"p -> w -> p"` — prompt optimization, then weight tuning, then prompt optimization again. The `optimizers` kwargs map letters to optimizer instances (`p=GEPA(...)`, `w=BootstrapFinetune(...)`). Defaults to `BootstrapRS` for `p` and `BootstrapFinetune` for `w`. Reach for it when prompt-only and weight-only have both been tried and you suspect they compose.

`dspy.Ensemble(reduce_fn=None, size=None, deterministic=False)` Not an optimizer in the usual sense — it composes already-compiled programs into one. `.compile(programs)` returns a module that runs each input through every program in parallel and reduces with `reduce_fn` (a custom function; defaults to majority voting via `dspy.majority`). Useful when several optimizer runs each produce a competent candidate and you want to combine them at inference time.

Specialized

`dspy.AvatarOptimizer(metric, max_iters=10, lower_bound=0, upper_bound=1, max_positive_inputs=10, max_negative_inputs=10, optimize_for="max")` Built for agent-style programs. Partitions the trainset by metric into positive examples (high scores) and negative examples (low scores). On each iteration, asks an LM to read positive and negative examples and propose instruction edits that explain the difference, then evaluates the new instructions. Niche but useful when the metric is a clean pass/fail and you need readable agent instructions.

Selection cheat sheet

Situation	Try
Just starting; no idea what helps	`BootstrapFewShot`
Demos vary in quality across attempts	`BootstrapFewShotWithRandomSearch`
Large trainset; inputs need different demos	`KNNFewShot`
Instructions look wrong; demos look fine	`COPRO` or `GEPA`
Both look weak; you have budget	`MIPROv2` or `GEPA`
Failure cases share a pattern you can name	`SIMBA` or `InferRules`
Prompt-only has plateaued; the model is tunable	`BootstrapFinetune`
You want to combine prompt + weight tuning	`BetterTogether`
You have multiple competent programs to combine	`Ensemble`
Agent / tool-use task	`AvatarOptimizer` or `GEPA`

Cross-links

- Metrics and evaluation — every optimizer compiles against a metric defined there; the `Prediction(score, feedback)` shape GEPA expects is documented there.
- GEPA in depth — the deep dive on the optimizer above.
- BootstrapFewShot family — the deep dive on `BootstrapFewShot` and its random-search variants.
- Fine-tuning with `BootstrapFinetune` — the deep dive on weight tuning.